In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
pd.set_option('display.max_columns', None)
pd.set_option('display.width', 120)
```

In [2]:
```
df = pd.read_csv('/content/bookbazaar_orders_2026.csv')
df.head(10)
```

Out[2]:

	Order_ID	Customer_ID	Customer_Name	Product_ID	Product	Category	Br
0	ORD0001	C212	Customer 212	B001	Python for Beginners	Technology	Brigh
1	ORD0002	C216	Customer 216	B012	Adventure Storybook	Children	Littlel
2	ORD0003	C221	Customer 221	B009	The Lost Map	Fiction	PageC
3	ORD0004	C209	Customer 209	B002	Introduction to Data Science	Technology	North
4	ORD0005	C206	Customer 206	B001	Python for Beginners	Technology	Brigh
5	ORD0006	C214	Customer 214	B007	Mindful Living	Self-Help	Sun Bo
6	ORD0007	C211	Customer 211	B011	Space Explorers	Children	Littlel
7	ORD0008	C231	NaN	B008	Build Better Habits	Self-Help	Sun Bo
8	ORD0009	C208	Customer 208	B010	Mountain Mystery	Fiction	North
9	ORD0010	C204	Customer 204	B007	Mindful Living	Self-Help	Sun Bo

In [3]:
```
df.shape
```

Out[3]:
```
(86, 19)
```

In [4]:
```
df.columns
```

Out[4]:
```
Index(['Order_ID', 'Customer_ID', 'Customer_Name', 'Product_ID', 'Product', 'Ca
tegory', 'Brand', 'Quantity',
       'Unit_Price', 'Discount_Percent', 'Shipping_Cost', 'Payment_Method', 'Or
der_Date', 'Ship_Date',
       'Delivery_Status', 'City', 'Region', 'Rating', 'Return_Status'],
      dtype='object')
```

In [5]:
```
df.dtypes
```

Out[5]:

	0
Order_ID	object
Customer_ID	object
Customer_Name	object
Product_ID	object
Product	object
Category	object
Brand	object
Quantity	int64
Unit_Price	object
Discount_Percent	float64
Shipping_Cost	float64
Payment_Method	object
Order_Date	object
Ship_Date	object
Delivery_Status	object
City	object
Region	object
Rating	float64
Return_Status	object

dtype: object

In [6]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 86 entries, 0 to 85
Data columns (total 19 columns):
 #   Column            Non-Null Count  Dtype  
---  ------            --------------  -----  
 0   Order_ID          86 non-null     object 
 1   Customer_ID       86 non-null     object 
 2   Customer_Name     85 non-null     object 
 3   Product_ID        86 non-null     object 
 4   Product           86 non-null     object 
 5   Category          86 non-null     object 
 6   Brand             86 non-null     object 
 7   Quantity          86 non-null     int64  
 8   Unit_Price        86 non-null     object 
 9   Discount_Percent  85 non-null     float64
 10  Shipping_Cost     85 non-null     float64
 11  Payment_Method    86 non-null     object 
 12  Order_Date        86 non-null     object 
 13  Ship_Date         84 non-null     object 
 14  Delivery_Status   86 non-null     object 
 15  City              86 non-null     object 
 16  Region            86 non-null     object 
 17  Rating            85 non-null     float64
 18  Return_Status     86 non-null     object 
dtypes: float64(3), int64(1), object(15)
memory usage: 12.9+ KB
```

In [7]:
```
df.describe()
```

Out[7]:

	Quantity	Discount_Percent	Shipping_Cost	Rating
count	86.000000	85.000000	85.000000	85.000000
mean	1.883721	7.176471	33.176471	4.023529
std	0.846009	5.948542	21.887796	0.899424
min	1.000000	0.000000	0.000000	2.000000
25%	1.000000	0.000000	30.000000	4.000000
50%	2.000000	5.000000	40.000000	4.000000
75%	2.750000	10.000000	50.000000	5.000000
max	4.000000	20.000000	60.000000	5.000000

In [8]:
```
df.isnull().sum()
```

Out[8]:

	0
Order_ID	0
Customer_ID	0
Customer_Name	1
Product_ID	0
Product	0
Category	0
Brand	0
Quantity	0
Unit_Price	0
Discount_Percent	1
Shipping_Cost	1
Payment_Method	0
Order_Date	0
Ship_Date	2
Delivery_Status	0
City	0
Region	0
Rating	1
Return_Status	0

dtype: int64

In [9]:
```
df.duplicated().sum(), df['Order_ID'].duplicated().sum()
```

Out[9]:
```
(np.int64(2), np.int64(2))
```

In [10]:
```
# Data Cleaning
```

In [11]:
```
text_cols = ['Customer_Name','Product','Category','Brand','Payment_Method',
             'Delivery_Status','City','Region','Return_Status']
for col in text_cols:
    df[col] = df[col].astype('string').str.strip()
```

In [12]:
```
df['City'] = df['City'].str.title()
df['Category'] = df['Category'].str.title()
df['Payment_Method'] = df['Payment_Method'].str.title().replace({'Upi':'UPI'})
df['Delivery_Status'] = df['Delivery_Status'].str.title()
```

In [13]:
```
numeric_cols = ['Quantity','Unit_Price','Discount_Percent','Shipping_Cost','Rati
for col in numeric_cols:
    df[col] = pd.to_numeric(df[col], errors='coerce')
```

In [14]:
```
df['Order_Date'] = pd.to_datetime(df['Order_Date'], errors='coerce')
df['Ship_Date'] = pd.to_datetime(df['Ship_Date'], errors='coerce')
df = df.drop_duplicates(subset='Order_ID', keep='first').copy()
```

In [15]:
```
df['Customer_Name'] = df['Customer_Name'].fillna('Unknown Customer')
df['Discount_Percent'] = df['Discount_Percent'].fillna(0)
df['Shipping_Cost'] = df['Shipping_Cost'].fillna(df['Shipping_Cost'].median())
df['Rating'] = df['Rating'].fillna(df['Rating'].median())
df = df.dropna(subset=['Order_Date','Quantity','Unit_Price']).copy()
```

In [16]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 82 entries, 0 to 83
Data columns (total 19 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   Order_ID          82 non-null     object
 1   Customer_ID       82 non-null     object
 2   Customer_Name     82 non-null     string
 3   Product_ID        82 non-null     object
 4   Product           82 non-null     string
 5   Category          82 non-null     string
 6   Brand             82 non-null     string
 7   Quantity          82 non-null     int64
 8   Unit_Price        82 non-null     float64
 9   Discount_Percent  82 non-null     float64
 10  Shipping_Cost     82 non-null     float64
 11  Payment_Method    82 non-null     string
 12  Order_Date        82 non-null     datetime64[ns]
 13  Ship_Date         80 non-null     datetime64[ns]
 14  Delivery_Status   82 non-null     string
 15  City              82 non-null     string
 16  Region            82 non-null     string
 17  Rating            82 non-null     float64
 18  Return_Status     82 non-null     string
dtypes: datetime64[ns](2), float64(4), int64(1), object(3), string(9)
memory usage: 12.8+ KB
```

In [17]:
```
df.isnull().sum()
```

Out[17]:

	0
Order_ID	0
Customer_ID	0
Customer_Name	0
Product_ID	0
Product	0
Category	0
Brand	0
Quantity	0
Unit_Price	0
Discount_Percent	0
Shipping_Cost	0
Payment_Method	0
Order_Date	0
Ship_Date	2
Delivery_Status	0
City	0
Region	0
Rating	0
Return_Status	0

dtype: int64

In [18]:
```
df.duplicated().sum(), df['Order_ID'].duplicated().sum()
```

Out[18]:
```
(np.int64(0), np.int64(0))
```

In [19]:
```
df.describe()
```

Out[19]:

	Quantity	Unit_Price	Discount_Percent	Shipping_Cost	Order_Date	Ship
count	82.000000	82.000000	82.000000	82.000000	82	
mean	1.902439	512.536585	7.195122	33.780488	2026-03-30 04:23:24.878048768	202 12
min	1.000000	249.000000	0.000000	0.000000	2026-01-02 00:00:00	202 00
25%	1.000000	299.000000	0.000000	30.000000	2026-02-11 12:00:00	202 12
50%	2.000000	449.000000	5.000000	40.000000	2026-03-28 12:00:00	202 00
75%	2.750000	749.000000	10.000000	50.000000	2026-05-12 06:00:00	202 18
max	4.000000	999.000000	20.000000	60.000000	2026-06-28 00:00:00	202 00
std	0.840484	234.738478	6.039866	21.978679	NaN	



Feature Engineering:

In [20]:
```
df['Gross_Amount'] = df['Quantity'] * df['Unit_Price']
df['Discount_Amount'] = df['Gross_Amount'] * df['Discount_Percent'] / 100
df['Net_Revenue'] = df['Gross_Amount'] - df['Discount_Amount']
df['Order_Total'] = df['Net_Revenue'] + df['Shipping_Cost']
```

In [21]:
```
df['Estimated_Cost'] = df['Gross_Amount'] * 0.70
df['Estimated_Profit'] = df['Net_Revenue'] - df['Estimated_Cost']
df['Order_Month'] = df['Order_Date'].dt.to_period('M').astype(str)
df['Order_Day'] = df['Order_Date'].dt.day_name()
df['Shipping_Days'] = (df['Ship_Date'] - df['Order_Date']).dt.days
```

In [22]:
```
df[['Order_ID','Product','Quantity','Unit_Price','Gross_Amount','Discount_Amount
```

Out[22]:

	Order_ID	Product	Quantity	Unit_Price	Gross_Amount	Discount_Amount	Net_F
0	ORD0001	Python for Beginners	2	799.0	1598.0	159.80	
1	ORD0002	Adventure Storybook	1	249.0	249.0	37.35	
2	ORD0003	The Lost Map	1	349.0	349.0	34.90	
3	ORD0004	Introduction to Data Science	1	999.0	999.0	0.00	
4	ORD0005	Python for Beginners	3	799.0	2397.0	359.55	



In [23]:
```
print('Total orders:', df['Order_ID'].nunique())
print('Unique customers:', df['Customer_ID'].nunique())
print('Unique products:', df['Product'].nunique())
print('Units sold:', df['Quantity'].sum())
print('Gross sales: Rs.', round(df['Gross_Amount'].sum(), 2))
```

```
Total orders: 82
Unique customers: 30
Unique products: 12
Units sold: 156
Gross sales: Rs. 81334.0
```

In [24]:
```
print('Total discount: Rs.', round(df['Discount_Amount'].sum(), 2))
print('Net ordered value: Rs.', round(df['Net_Revenue'].sum(), 2))
print('Average order value: Rs.', round(df['Order_Total'].mean(), 2))
print('Estimated profit: Rs.', round(df['Estimated_Profit'].sum(), 2))
```

```
Total discount: Rs. 6189.4
Net ordered value: Rs. 75144.6
Average order value: Rs. 950.18
Estimated profit: Rs. 18210.8
```

In [25]:
```
product_summary = df.groupby('Product').agg(
    Orders=('Order_ID','nunique'), Units_Sold=('Quantity','sum'),
    Net_Revenue=('Net_Revenue','sum'), Average_Rating=('Rating','mean')
).sort_values('Net_Revenue', ascending=False)
product_summary
```

Out[25]:

Product	Orders	Units_Sold	Net_Revenue	Average_Rating
Python for Beginners	8	19	13742.80	4.000000
Introduction to Data Science	7	14	13086.90	4.285714
World History Made Easy	7	14	9137.80	4.142857
Build Better Habits	10	20	7700.70	4.100000
Space Explorers	12	28	7435.35	3.750000
Mindful Living	9	13	5545.15	4.333333
Ancient India	6	9	4971.70	3.833333
Mountain Mystery	9	14	3767.40	3.888889
Indian Freedom Movement	3	7	3343.30	4.333333
Excel and Analytics	3	5	2985.40	4.000000
Adventure Storybook	5	8	1805.25	3.800000
The Lost Map	3	5	1622.85	3.333333

In [26]:
```
category_summary = df.groupby('Category').agg(
    Orders=('Order_ID','nunique'), Units=('Quantity','sum'),
    Revenue=('Net_Revenue','sum'), Estimated_Profit=('Estimated_Profit','sum')
).sort_values('Revenue', ascending=False)
category_summary
```

Out[26]:

Category	Orders	Units	Revenue	Estimated_Profit
Technology	19	40	31088.40	7351.40
History	15	28	16179.50	3669.10
Self-Help	19	33	13245.85	3573.95
Children	17	36	9240.60	2377.80
Fiction	12	19	5390.25	1238.55

In [27]:
```
region_summary = df.groupby(['Region','City']).agg(
    Orders=('Order_ID','nunique'), Revenue=('Net_Revenue','sum'),
    Units=('Quantity','sum')
)
region_summary
```

Out[27]:

Region	City	Orders	Revenue	Units
East	Bhubaneswar	6	8113.55	12
	Kolkata	6	6444.15	12
North	Delhi	14	10418.90	24
	Jaipur	3	2389.65	8
South	Bengaluru	13	14298.10	33
	Chennai	21	18329.25	33
	Hyderabad	4	2397.50	8
West	Mumbai	7	5982.10	12
	Pune	8	6771.40	14

In [28]:
```
payment_summary = df.groupby('Payment_Method').agg(
    Orders=('Order_ID','nunique'), Revenue=('Order_Total','sum')
).sort_values('Orders', ascending=False)
payment_summary
```

Out[28]:

Payment_Method	Orders	Revenue
UPI	34	30853.95
Debit Card	19	20324.05
Credit Card	17	16890.00
Cash On Delivery	7	4340.45
Net Banking	5	5506.15

In [29]:
```
monthly_summary = df.groupby('Order_Month').agg(
    Orders=('Order_ID','nunique'), Units=('Quantity','sum'),
    Net_Revenue=('Net_Revenue','sum'), Average_Order_Value=('Order_Total','mean'
)
monthly_summary
```

Out[29]:

Order_Month	Orders	Units	Net_Revenue	Average_Order_Value
2026-01	14	23	11672.30	863.735714
2026-02	14	29	16405.00	1216.071429
2026-03	14	27	13251.65	975.832143
2026-04	13	31	12203.40	971.030769
2026-05	13	19	9437.80	764.446154
2026-06	14	27	12174.45	898.175000

In [30]:
```
customer_summary = df.groupby(['Customer_ID','Customer_Name']).agg(
    Orders=('Order_ID','nunique'), Total_Spent=('Order_Total','sum'),
    Units=('Quantity','sum')
).sort_values('Total_Spent', ascending=False)
customer_summary.head(10)
```

Out[30]:

Customer_ID	Customer_Name	Orders	Total_Spent	Units
C215	**Customer 215**	4	7060.70	10
C212	**Customer 212**	5	5998.95	9
C216	**Customer 216**	5	4748.85	9
C211	**Customer 211**	4	3848.65	9
C223	**Customer 223**	2	3790.25	5
C224	**Customer 224**	3	3674.10	6
C208	**Customer 208**	2	3554.60	6
C207	**Customer 207**	4	3041.10	9
C227	**Customer 227**	2	2796.30	4
C225	**Customer 225**	3	2690.35	5

In [31]:
```
product_summary[product_summary['Net_Revenue'] > product_summary['Net_Revenue'].
```

Out[31]:

Product	Orders	Units_Sold	Net_Revenue	Average_Rating
Python for Beginners	8	19	13742.80	4.000000
Introduction to Data Science	7	14	13086.90	4.285714
World History Made Easy	7	14	9137.80	4.142857
Build Better Habits	10	20	7700.70	4.100000
Space Explorers	12	28	7435.35	3.750000

In [32]:
```
df['Delivery_Status'].value_counts()
```

Out[32]:

Delivery_Status	count
Delivered	64
Returned	9
Shipped	7
Cancelled	2

dtype: Int64

In [33]:
```
df.groupby('Category').agg(
    Average_Discount=('Discount_Percent','mean'),
    Discount_Amount=('Discount_Amount','sum'),
    Net_Revenue=('Net_Revenue','sum')
).sort_values('Net_Revenue', ascending=False)
```

Out[33]:

Category	Average_Discount	Discount_Amount	Net_Revenue
Technology	7.631579	2821.60	31088.40
History	8.333333	1692.50	16179.50
Self-Help	5.263158	571.15	13245.85
Children	7.352941	563.40	9240.60
Fiction	7.916667	540.75	5390.25

In [34]:
```
df[df['Discount_Percent'] >= 10][['Order_ID','Product','Discount_Percent','Net_R
```

Out[34]:

	Order_ID	Product	Discount_Percent	Net_Revenue
0	ORD0001	Python for Beginners	10.0	1438.20
1	ORD0002	Adventure Storybook	15.0	211.65
2	ORD0003	The Lost Map	10.0	314.10
4	ORD0005	Python for Beginners	15.0	2037.45
5	ORD0006	Mindful Living	20.0	359.20
6	ORD0007	Space Explorers	10.0	753.30
7	ORD0008	Build Better Habits	15.0	339.15
8	ORD0009	Mountain Mystery	10.0	807.30
11	ORD0012	Python for Beginners	10.0	1438.20
13	ORD0014	Space Explorers	10.0	251.10

In [35]:
```
df.groupby('Product').agg(
    Average_Rating=('Rating','mean'), Rating_Count=('Rating','count')
).sort_values('Average_Rating', ascending=False)
```

Out[35]:

Product	Average_Rating	Rating_Count
Mindful Living	4.333333	9
Indian Freedom Movement	4.333333	3
Introduction to Data Science	4.285714	7
World History Made Easy	4.142857	7
Build Better Habits	4.100000	10
Python for Beginners	4.000000	8
Excel and Analytics	4.000000	3
Mountain Mystery	3.888889	9
Ancient India	3.833333	6
Adventure Storybook	3.800000	5
Space Explorers	3.750000	12
The Lost Map	3.333333	3

In [36]:
```
df.groupby('Customer_ID')['Order_ID'].nunique().sort_values(ascending=False).hea
```

Out[36]:

Customer_ID	Order_ID
C204	5
C212	5
C216	5
C215	4
C211	4
C214	4
C231	4
C207	4
C210	3
C203	3

dtype: int64

In [37]:
```
pd.crosstab(df['Payment_Method'], df['Delivery_Status'])
```

Out[37]:

Payment_Method	Cancelled	Delivered	Returned	Shipped
Cash On Delivery	0	4	2	1
Credit Card	1	13	2	1
Debit Card	1	17	0	1
Net Banking	0	4	1	0
UPI	0	26	4	4

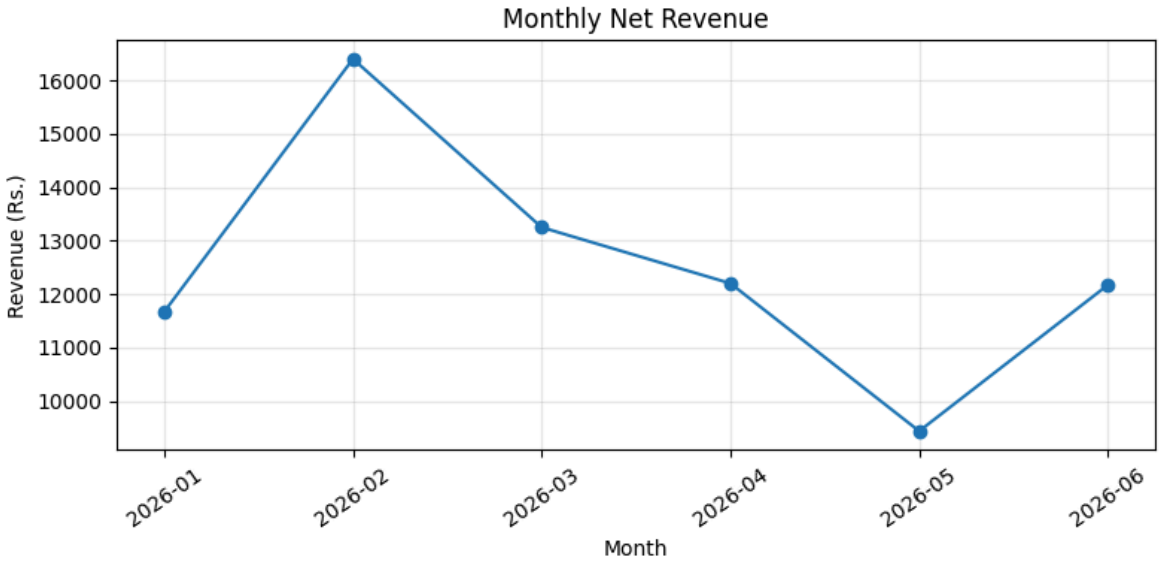
In [38]:
```
numeric = ['Quantity','Unit_Price','Discount_Percent','Shipping_Cost',
           'Gross_Amount','Net_Revenue','Order_Total','Estimated_Profit','Rating
df[numeric].corr()
```

Out[38]:

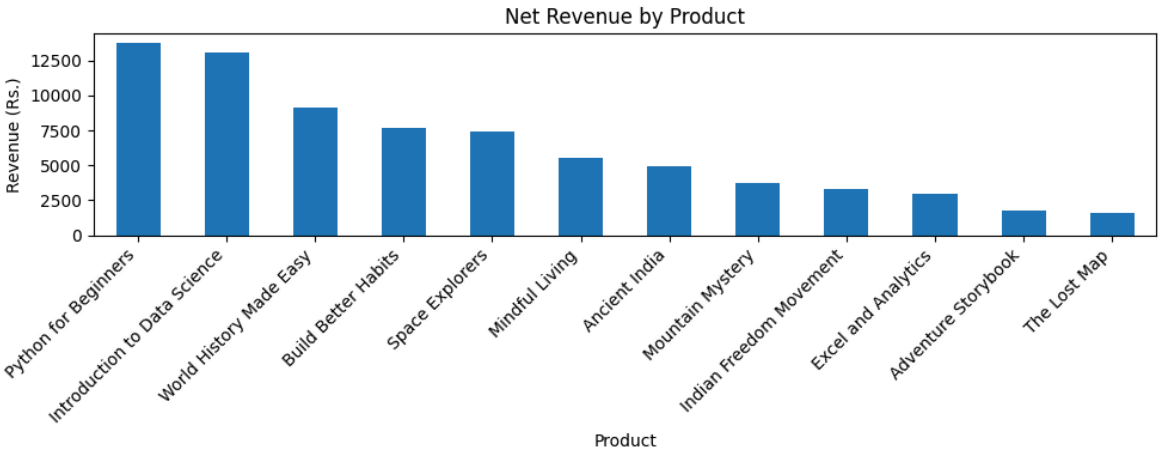
	Quantity	Unit_Price	Discount_Percent	Shipping_Cost	Gross_Amount
Quantity	1.000000	0.086247	-0.030251	0.066995	0.659795
Unit_Price	0.086247	1.000000	0.061941	0.247916	0.754293
Discount_Percent	-0.030251	0.061941	1.000000	0.136666	0.102069
Shipping_Cost	0.066995	0.247916	0.136666	1.000000	0.202833
Gross_Amount	0.659795	0.754293	0.102069	0.202833	1.000000
Net_Revenue	0.668528	0.753269	0.019268	0.202804	0.995276
Order_Total	0.665670	0.756206	0.023996	0.236916	0.994702
Estimated_Profit	0.653169	0.701121	-0.247549	0.189553	0.915560
Rating	-0.016525	0.136677	-0.011498	-0.006319	0.080206

Visualizations:

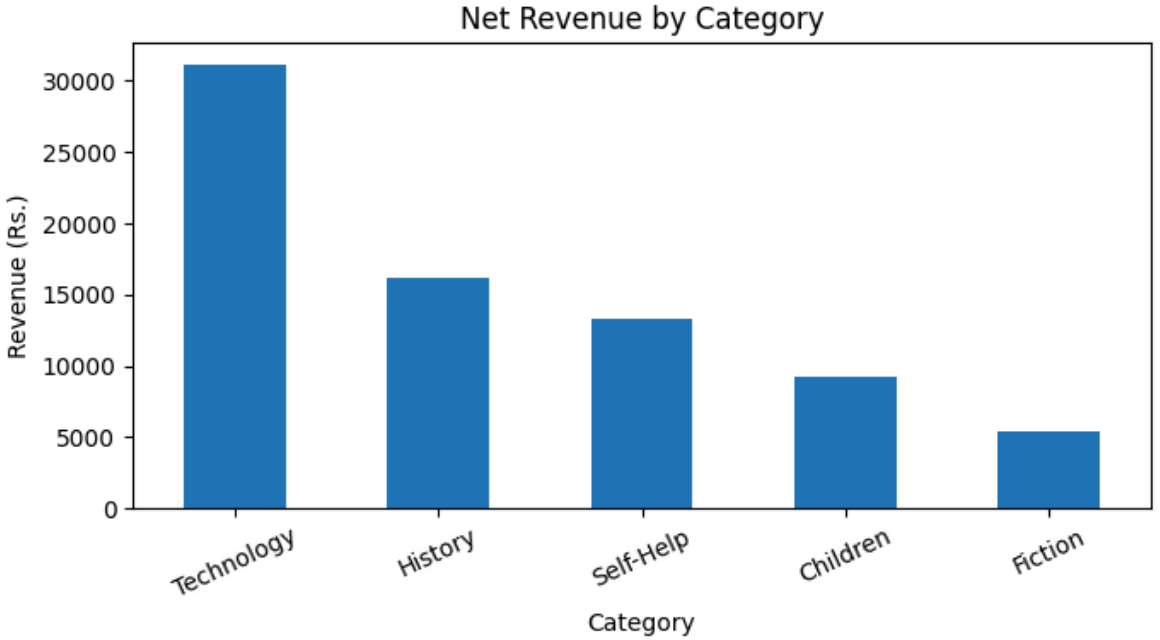
In [39]:
```
# Plot 1: Monthly net revenue
monthly_revenue = df.groupby('Order_Month')['Net_Revenue'].sum()
plt.figure(figsize=(8,4))
plt.plot(monthly_revenue.index, monthly_revenue.values, marker='o')
plt.title('Monthly Net Revenue'); plt.xlabel('Month'); plt.ylabel('Revenue (Rs.)
plt.xticks(rotation=35); plt.grid(alpha=0.3); plt.tight_layout(); plt.show()
```



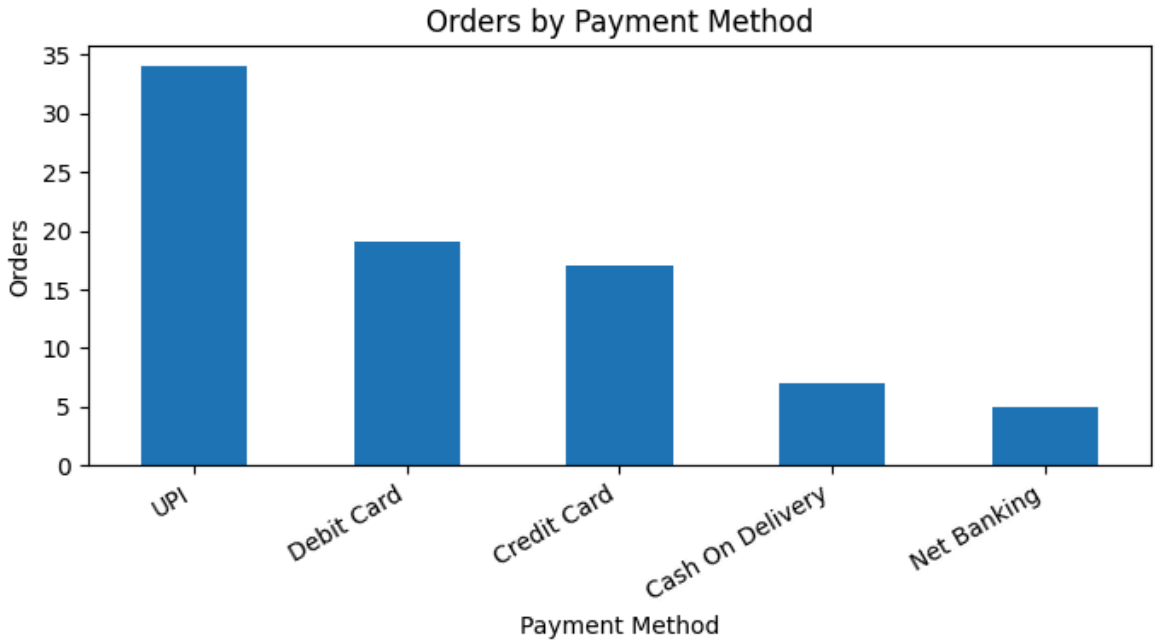
In [40]:
```
# Plot 2: Net revenue by product
product_revenue = df.groupby('Product')['Net_Revenue'].sum().sort_values(ascendi
plt.figure(figsize=(10,4))
product_revenue.plot(kind='bar')
plt.title('Net Revenue by Product'); plt.xlabel('Product'); plt.ylabel('Revenue
plt.xticks(rotation=45, ha='right'); plt.tight_layout(); plt.show()
```



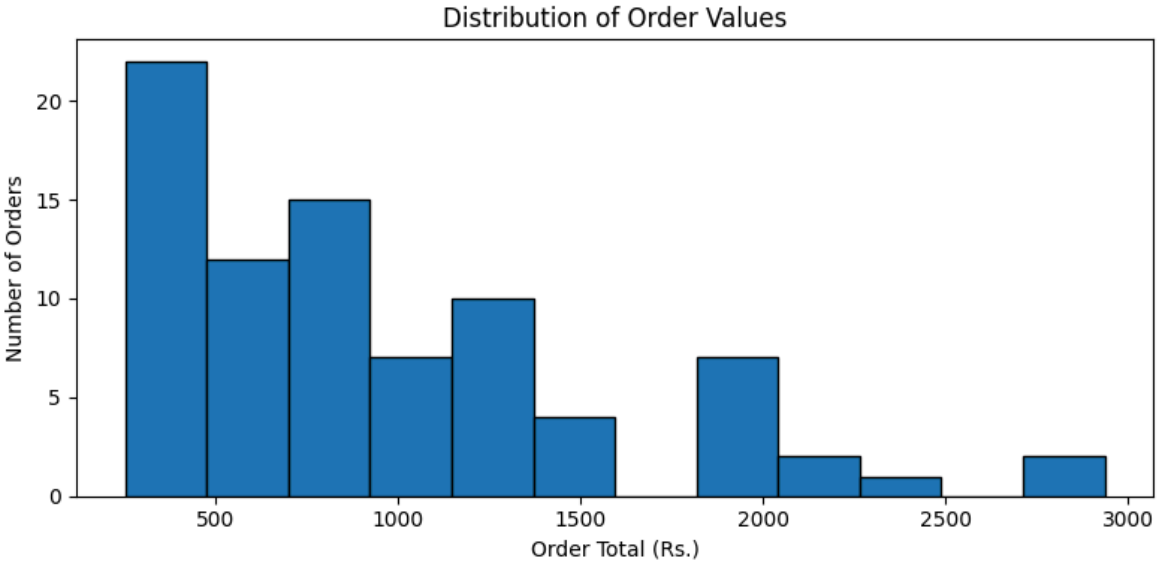
In [41]:
```
# Plot 3: Net revenue by category
category_revenue = df.groupby('Category')['Net_Revenue'].sum().sort_values(ascen
plt.figure(figsize=(7,4))
category_revenue.plot(kind='bar')
plt.title('Net Revenue by Category'); plt.xlabel('Category'); plt.ylabel('Revenu
plt.xticks(rotation=25); plt.tight_layout(); plt.show()
```



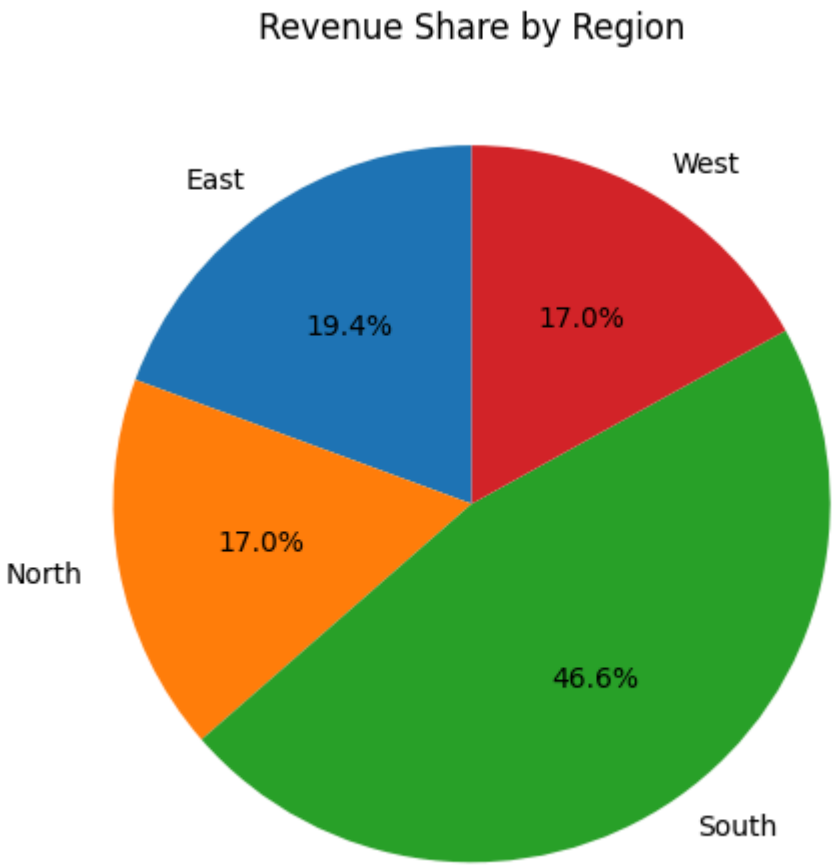
In [42]:
```
# Plot 4: Orders by payment method
payment_counts = df['Payment_Method'].value_counts()
plt.figure(figsize=(7,4))
payment_counts.plot(kind='bar')
plt.title('Orders by Payment Method'); plt.xlabel('Payment Method'); plt.ylabel(
plt.xticks(rotation=30, ha='right'); plt.tight_layout(); plt.show()
```



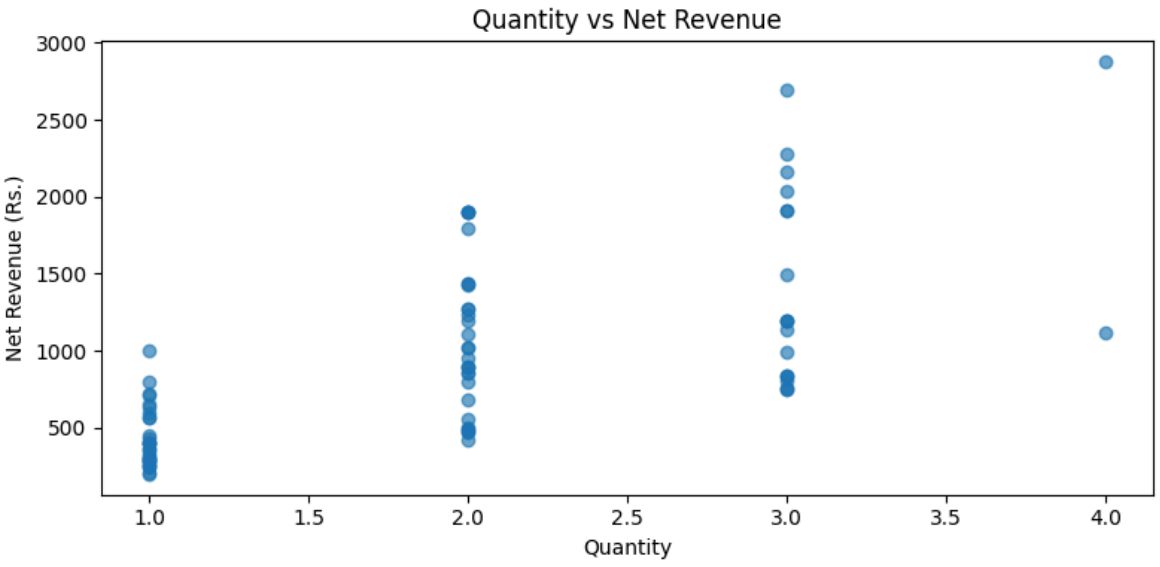
In [43]:
```
# Plot 5: Distribution of order values
plt.figure(figsize=(8,4))
plt.hist(df['Order_Total'], bins=12, edgecolor='black')
plt.title('Distribution of Order Values')
plt.xlabel('Order Total (Rs.)'); plt.ylabel('Number of Orders')
plt.tight_layout(); plt.show()
```



In [44]:
```
# Plot 6: Revenue share by region
region_revenue = df.groupby('Region')['Net_Revenue'].sum()
plt.figure(figsize=(7,5))
plt.pie(region_revenue, labels=region_revenue.index, autopct='%1.1f%%', startang
plt.title('Revenue Share by Region')
plt.tight_layout(); plt.show()
```

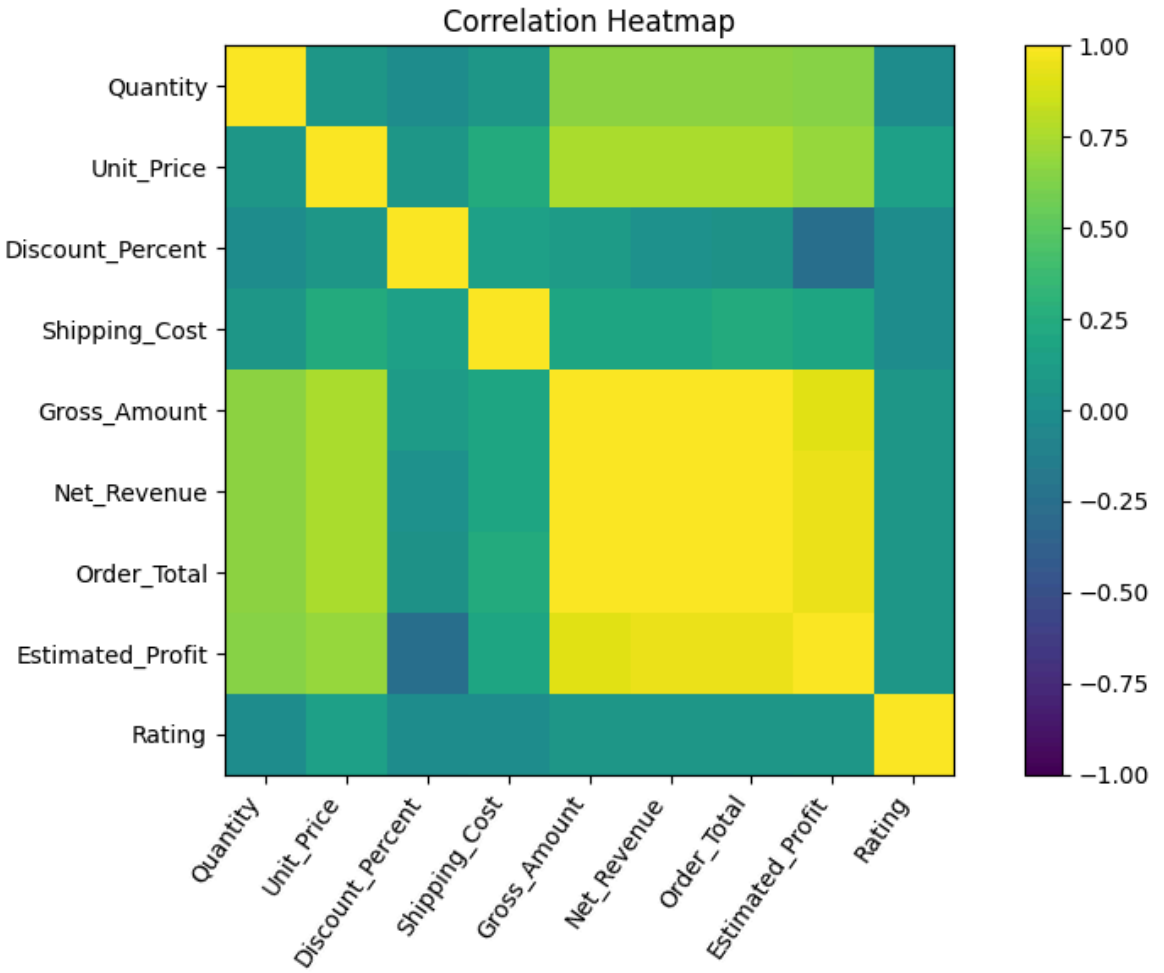


In [45]:
```
# Plot 7: Quantity vs net revenue
plt.figure(figsize=(8,4))
plt.scatter(df['Quantity'], df['Net_Revenue'], alpha=0.65)
plt.title('Quantity vs Net Revenue')
plt.xlabel('Quantity'); plt.ylabel('Net Revenue (Rs.)')
plt.tight_layout(); plt.show()
```

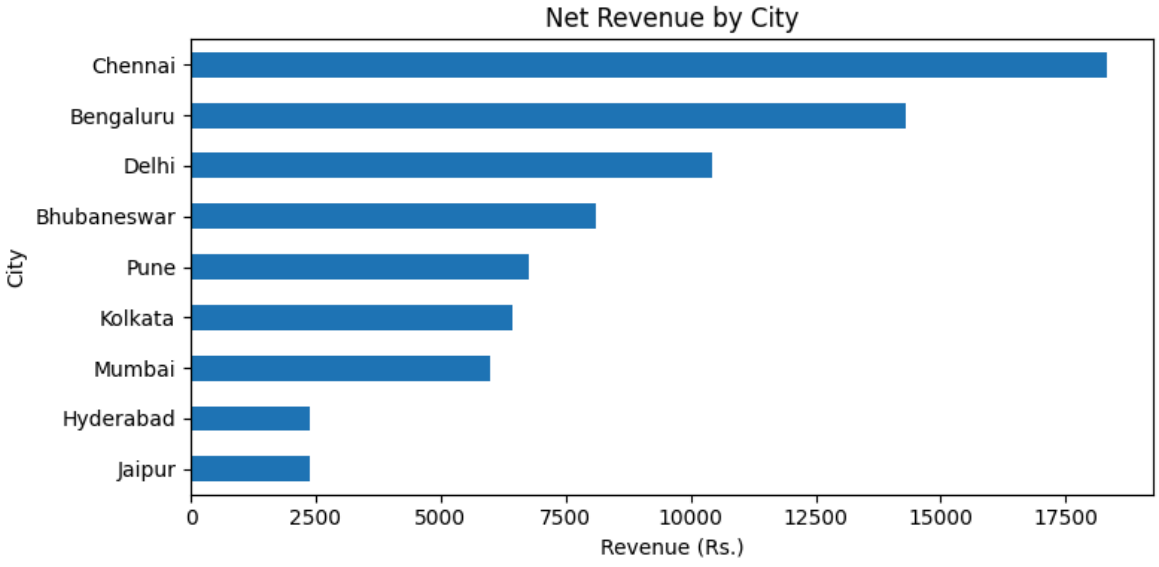


In [46]:
```
# Plot 8: Correlation heatmap
corr = df[numeric].corr()
plt.figure(figsize=(9,6))
plt.imshow(corr, cmap='viridis', vmin=-1, vmax=1)
plt.xticks(range(len(corr.columns)), corr.columns, rotation=55, ha='right')
```

```
plt.yticks(range(len(corr.columns)), corr.columns)
plt.title('Correlation Heatmap'); plt.colorbar(); plt.tight_layout(); plt.show()
```

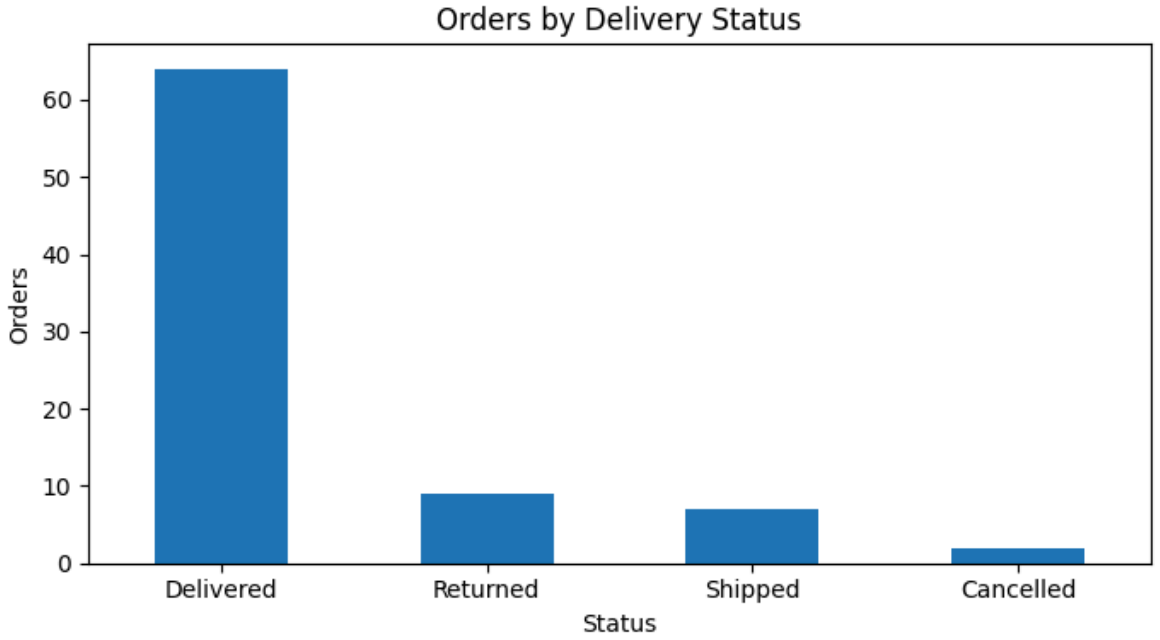


In [47]:
```
#  Revenue by city
city_revenue = df.groupby('City')['Net_Revenue'].sum().sort_values()
plt.figure(figsize=(8,4))
city_revenue.plot(kind='barh')
plt.title('Net Revenue by City'); plt.xlabel('Revenue (Rs.)')
plt.tight_layout(); plt.show()
```

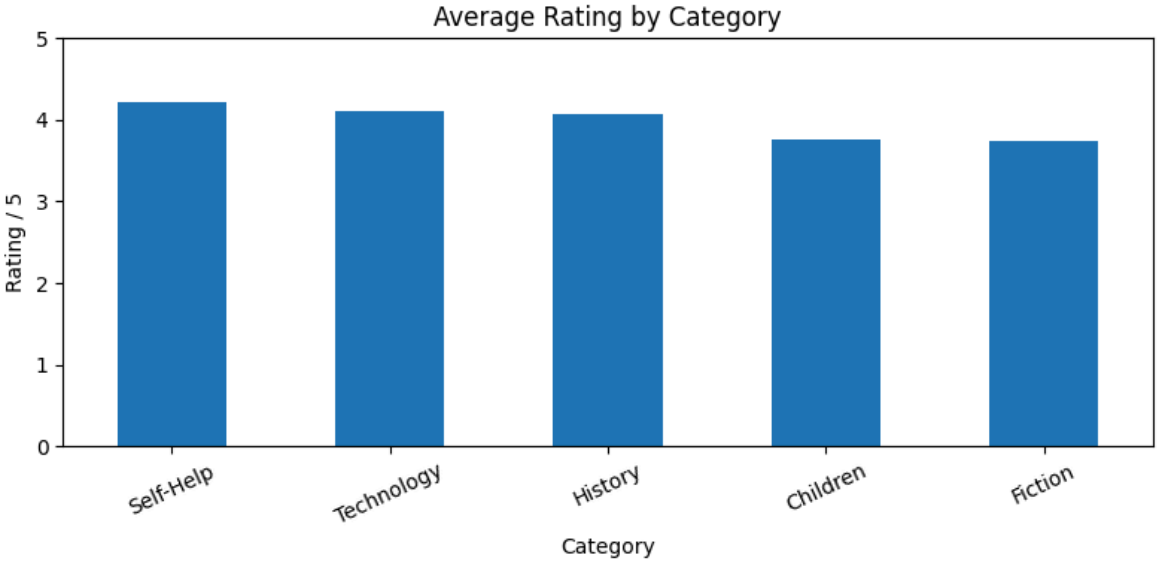


In [48]:
```
#  Delivery status
status_counts = df['Delivery_Status'].value_counts()
```

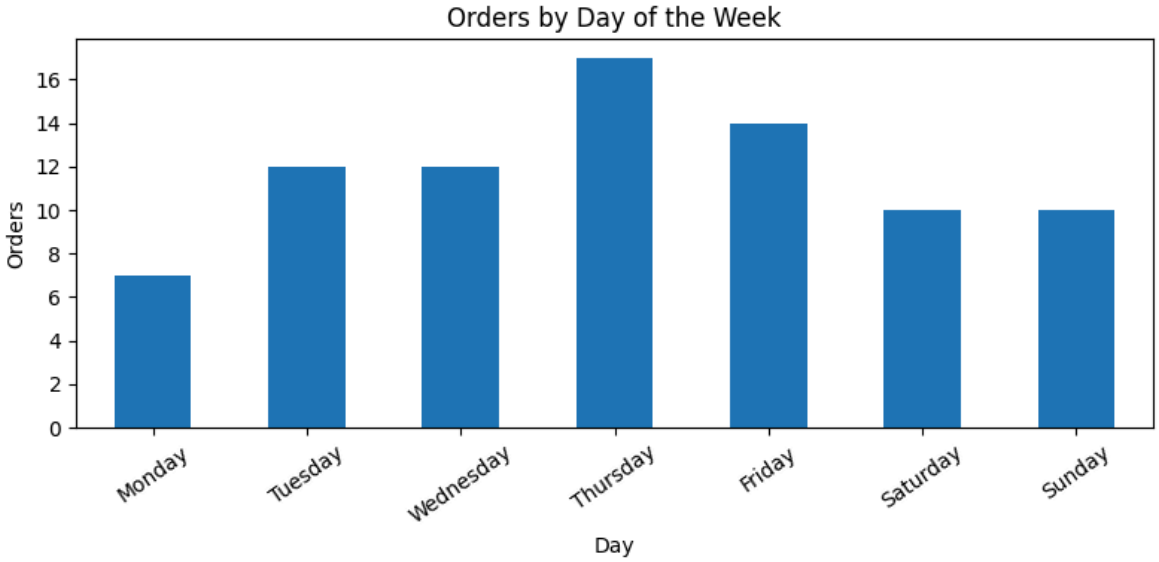
```
plt.figure(figsize=(7,4))
status_counts.plot(kind='bar')
plt.title('Orders by Delivery Status'); plt.xlabel('Status'); plt.ylabel('Orders
plt.xticks(rotation=0); plt.tight_layout(); plt.show()
```



In [49]:
```
# Average rating by category
category_rating = df.groupby('Category')['Rating'].mean().sort_values(ascending=
plt.figure(figsize=(8,4))
category_rating.plot(kind='bar')
plt.title('Average Rating by Category'); plt.xlabel('Category'); plt.ylabel('Rat
plt.ylim(0,5); plt.xticks(rotation=25); plt.tight_layout(); plt.show()
```



In [50]:
```
# Orders by weekday
day_order = ['Monday','Tuesday','Wednesday','Thursday','Friday','Saturday','Sund
weekday_orders = df['Order_Day'].value_counts().reindex(day_order, fill_value=0)
plt.figure(figsize=(8,4))
weekday_orders.plot(kind='bar')
plt.title('Orders by Day of the Week'); plt.xlabel('Day'); plt.ylabel('Orders')
plt.xticks(rotation=35); plt.tight_layout(); plt.show()
```


Orders by Day of the Week
Orders
0
2
4
6
8
10
12
14
16
Monday
Tuesday
Wednesday
Thursday
Friday
Saturday
Sunday
Day